

Software Requirements Specification

for

Disaster-Response Ground Scout: An Autonomous Robot for Human Presence and Infrastructure Hazard Detection

Prepared in accordance with IEEE Std 830-1998

Version 1.0

Date: September 12, 2026

Authors:

Mashal Asif

Table Of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions, Acronyms, and Abbreviations
- 1.4 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Characteristics
- 2.4 Constraints
- 2.5 Assumptions and Dependencies
- 2.6 Apportioning of Requirements

3. Specific Requirements

- 3.1 Interface Requirements
 - 3.1.1 User Interfaces
 - 3.1.2 Hardware Interfaces
 - 3.1.3 Software Interfaces
 - 3.1.4 Communication Interfaces
- 3.2 Functional Requirements
 - 3.2.1 Human Detection Module
 - 3.2.2 Hazard Detection Module
 - 3.2.3 Obstacle Avoidance & Motor Control Module
 - 3.2.4 Wireless Streaming & Monitoring Module
- 3.3 Performance Requirements
- 3.4 Design Constraints
- 3.5 Software System Attributes

4. Prioritization and Development Plan

- 4.1 Choice of Prioritization Method
- 4.2 Weekly Build Milestones

Appendix I: Hardware Bill of Materials

Appendix II: Test Cases for Mock Disaster-Zone Trials

1. Introduction

1.1 Purpose

This document specifies the software and hardware requirements for SCOUT, an autonomous ground robot designed to assist in disaster-response scenarios by detecting human presence and identifying infrastructure hazards such as exposed wiring. It is intended to guide the design, development, and testing phases of the project, and to serve as a reference for evaluating whether the final system meets its intended objectives.

1.2 Scope

SCOUT is a wheeled, autonomous ground robot that navigates a disaster or conflict-affected area, streams live camera footage, and uses onboard AI to detect the presence of humans and visual hazards (e.g., exposed wiring). The system does not perform physical rescue operations; its function is limited to detection, navigation, and reporting. This project is developed as a semester-long lab prototype and is tested in a controlled, simulated disaster environment rather than a real disaster zone.

1.3 Definitions, Acronyms, and Abbreviations

- SRS – Software Requirements Specification
- AI – Artificial Intelligence
- TFLite – TensorFlow Lite (used for on-device AI inference)
- ESP32-CAM – Microcontroller module with integrated camera and WiFi
- ATmega328P – Microcontroller used on the Arduino Nano for motor and sensor control
- PWM – Pulse Width Modulation (used for motor speed control)
- SAR – Search and Rescue

1.4 References

- ESP32-CAM official documentation (Espressif Systems)
- Arduino/ATmega328P datasheet (Microchip)
- TensorFlow Lite for Microcontrollers documentation

2. Overall Description

2.1 Product Perspective

SCOUT is a standalone, self-contained robotic system. It is not part of a larger product line and does not depend on any existing software system. It consists of two microcontroller boards (ESP32-CAM and Arduino Nano) working together, along with motors, sensors, and a camera, all mounted on a wheeled chassis.

2.2 Product Functions

At a high level, SCOUT will:

- Move autonomously while avoiding obstacles
- Capture live video using an onboard camera
- Detect human presence using an AI model
- Detect visual hazards such as exposed wiring
- Stream video and alerts wirelessly to a connected device

2.3 User Characteristics

The intended users of SCOUT are disaster-response personnel or rescue teams who may not have technical expertise in robotics or AI. The system is designed to require minimal training — users only need to power on the robot and monitor the live feed/alerts on a connected device.

2.4 Constraints

- The system must be developed within a limited student budget (~\$50)
- The project must be completed within a 10-week development timeline
- Hardware is limited to components available through local suppliers in Pakistan
- Processing power is constrained by the capabilities of the ESP32-CAM and Arduino Nano (no high-performance onboard computer)
- The system is tested in a controlled, simulated disaster environment rather than a real disaster zone

2.5 Assumptions and Dependencies

- It is assumed that a stable WiFi connection is available for live video streaming during testing

- It is assumed that lighting conditions during testing are sufficient for the camera and AI model to function reliably
- The project depends on the availability of pre-trained AI models (e.g., TensorFlow Lite) that can be adapted for use on the ESP32-CAM
- It is assumed that the test environment (mock disaster zone) reasonably approximates the visual conditions the system is designed to detect

2.6 Apportioning of Requirements

Some features are considered essential for the current version of SCOUT (e.g., human detection, basic obstacle avoidance), while others are considered enhancements that may be deferred to future versions if time or resources do not allow (e.g., advanced hazard classification, multi-hazard detection, real-time mapping of detected hazards/humans).

3. Specific Requirements

3.1 Interface Requirements

3.1.1 User Interfaces

The user interacts with SCOUT through a simple web-based dashboard (accessed via browser on a phone or laptop), which displays the live camera feed and shows alerts when a human or hazard is detected. No installation of additional software is required on the user's end.

3.1.2 Hardware Interfaces

- ESP32-CAM interfaces with the onboard camera module for image capture
- Arduino Nano interfaces with the L298N motor driver to control two DC motors
- Arduino Nano interfaces with the HC-SR04 ultrasonic sensor via digital I/O pins for distance measurement
- ESP32-CAM and Arduino Nano communicate with each other via serial (UART) connection

3.1.3 Software Interfaces

- Arduino IDE is used to write and upload firmware to both the ESP32-CAM and Arduino Nano
- TensorFlow Lite for Microcontrollers is used to run the human/hazard detection AI model on the ESP32-CAM
- A basic web server (hosted on the ESP32-CAM) serves the live video feed and status dashboard to the user's browser

3.1.4 Communication Interfaces

- Wi-Fi is used for wireless video streaming and alert notifications between SCOUT and the user's device
- Serial (UART) communication is used internally between the ESP32-CAM and Arduino Nano for coordinating detection results with motor control

3.2 Functional Requirements

3.2.1 Human Detection Module

The system shall capture live video frames and process them using an onboard AI model to detect the presence of a human in the frame. When a human is detected, the system shall trigger an alert and log the detection with a timestamp.

3.2.2 Hazard Detection Module

The system shall analyse captured frames to identify visual indicators of infrastructure hazards (e.g., exposed wiring based on colour/shape patterns). When a hazard is detected, the system shall trigger an alert distinct from the human-detection alert.

3.2.3 Obstacle Avoidance & Motor Control Module

The system shall continuously measure distance to nearby objects using an ultrasonic sensor. If an object is detected within a defined threshold distance, the system shall automatically stop and redirect the robot to avoid collision, without requiring user intervention.

3.2.4 Wireless Streaming & Monitoring Module

The system shall stream live video over Wi-Fi to a connected device in real time. It shall also transmit alert notifications (human detected / hazard detected) to the same interface as they occur.

3.3 Performance Requirements

- The system shall process each camera frame for human/hazard detection within an acceptable delay (e.g., under 2 seconds per frame) to allow near real-time operation
- The robot shall respond to obstacle detection and stop within a safe distance (e.g., 10-15 cm) to avoid collision
- The video stream shall maintain a usable frame rate for monitoring purposes, even if not high-definition

3.4 Design Constraints

- The system must operate using only the processing capability of the ESP32-CAM and Arduino Nano (no external server-side processing for core detection functions)
- The AI model used must be lightweight enough to run on embedded hardware (quantized/optimized for microcontroller deployment)
- Total hardware cost must remain within the allocated project budget (~\$50)
- The system must be built and tested within the 10-week project timeline

3.5 Software System Attributes

- Reliability: The system should consistently detect humans and hazards under the lighting/visual conditions of the test environment
- Maintainability: Code should be modular (separate logic for detection, motor control, and communication) to allow easy debugging and future upgrades
- Portability: Firmware should be adaptable to similar ESP32/Arduino-based hardware with minimal modification
- Usability: The monitoring dashboard should be simple enough for a non-technical user to understand alerts without additional training

4. Prioritization and Development Plan

4.1 Choice of Prioritization Method

This project uses the MoSCoW method (Must have, Should have, Could have, Won't have) to prioritize requirements, given the fixed 10-week timeline and limited budget. This method was chosen because it clearly separates features that are essential for a working demo from those that are enhancements, allowing scope to be adjusted safely if time runs short without jeopardizing the core project goals.

- Must have: Autonomous movement, obstacle avoidance, human detection, wireless video streaming
- Should have: Hazard detection (exposed wiring), alert logging with timestamps
- Could have: Web dashboard with alert history, gas/smoke sensor integration
- Won't have (this version): Real-time mapping, multi-hazard classification, outdoor/real disaster-zone deployment

4.2 Weekly Build Milestones

- Week 1: Development environment setup — Arduino IDE, ESP32 board support, review of AI model options for human detection. Order hardware in parallel (delivery time runs concurrently with software prep).
- Week 2: Study and adapt a pre-trained human-detection model (TensorFlow Lite) for ESP32-CAM; write/test detection logic in isolation (using sample images) while hardware is in transit.
- Week 3: Write obstacle-avoidance logic and motor-control code for Arduino Nano (logic can be dry-run/simulated conceptually before hardware arrives); begin hazard-detection logic design.
- Week 4: Hardware arrives — assemble chassis, wire motors/sensors, upload and test previously written motor-control and obstacle-avoidance code.
- Week 5: Deploy and test human-detection model on actual ESP32-CAM hardware; debug real-world performance vs. earlier test-image results.
- Week 6: Implement and test hazard detection (exposed wiring) on real hardware.
- Week 7: Full system integration — movement + detection + alerts working together.
- Week 8: Testing in mock disaster-zone environment.
- Week 9: Debugging and refinement based on test results.
- Week 10: Final documentation and demo preparation.

Appendix I: Hardware Bill of Materials

#	Component	Purpose	Approx. Cost (USD)	Approx. Cost (PKR)
1	ESP32-CAM module	Camera + AI inference + WiFi	\$6-8	Rs1,660-2,215

2	FTDI USB-to-Serial programmer	Uploading code to ESP32-CAM	\$2-3	Rs555-830
3	Arduino Nano (ATmega328P)	Motor and sensor control	\$3-4	Rs830-1,110
4	DC gear motors (TT motors) x2	Robot movement	\$4-6	Rs1,110-1,660
5	Wheels x2	Movement	\$1-2	Rs275-555
6	2WD robot chassis kit	Structural base	\$4-6	Rs1,110-1,660
7	L298N motor driver	Motor speed/direction control	\$2-3	Rs555-830
8	HC-SR04 ultrasonic sensor	Obstacle detection	\$1-2	Rs275-555
9	18650 Li-ion batteries + holder	Power supply	\$4-6	Rs1,110-1,660
10	Jumper wires (pack)	Circuit connections	\$1-2	Rs275-555
11	Breadboard	Prototyping	\$1	Rs275
12	On/off switch	Power control	\$0.5	Rs140
13	MQ-2 gas/smoke sensor (optional)	Additional hazard detection	\$2	Rs555

Total estimated cost: ~\$32-45 USD ≈ Rs8,900-12,500 PKR

Appendix II: Test Cases for Mock Disaster-Zone Trials

Test ID	Test Description	Input/Condition	Expected Output	Pass/Fail
TC-01	Human detection accuracy	Dummy figure placed in camera's field of view	System flags "human detected" alert	
TC-02	Human detection at different angles	Dummy figure at 0°, 45°, 90° relative to camera	System detects human at all tested angles	
TC-03	Human detection with partial occlusion	Dummy figure partially hidden behind debris	System detects human or logs reduced confidence appropriately	
TC-04	Human detection at varying distances	Dummy figure placed at 0.5m, 1m, 2m from camera	System detects human within the tested range; note max reliable distance	
TC-05	Hazard detection (exposed wire)	Colored wire placed within camera's field of view	System flags "hazard detected" alert	
TC-06	Hazard detection false-positive check	Similarly colored non-hazard object placed in view	System does not incorrectly flag it as a hazard	
TC-07	Obstacle avoidance — direct path	Object placed directly in robot's path	Robot stops/redirects before collision	
TC-08	Obstacle avoidance — distance accuracy	Object placed at 5cm, 15cm, 30cm from robot	Robot reacts consistently at the defined threshold distance	
TC-09	Obstacle avoidance — angled approach	Object placed at an angle to robot's path	Robot detects and avoids without direct head-on alignment	
TC-10	Video streaming stability	Robot moving while streaming video	Live feed remains viewable without significant lag/drop	
TC-11	Alert notification delivery	Human or hazard detected during a run	Alert appears on connected device/dashboard in near real-time	
TC-12	Combined detection scenario	Both human and hazard present in same test run	System correctly distinguishes and flags both, without confusing one for the other	
TC-13	Battery/power reliability	Continuous operation over a set duration (e.g., 15-20 min)	System remains powered and functional throughout	

TC-14	No-target false positive check	No human/hazard present in test environment	System does not incorrectly trigger any alerts
TC-15	Recovery after obstacle stop	Robot stops due to obstacle, obstacle then removed	Robot resumes autonomous movement without manual reset